SOFTWARE ENGINEERING PROJECT DOCUMENTATION

Voice-Based Concept Understanding Analyser

(VBCUA)

An AI-Powered Speech and Semantic Evaluation System for Assessing Conceptual Understanding

Attribute	Details
Project Title	Voice-Based Concept Understanding Analyser (VBCUA)
Team ID	SWTID-2026-9346
Project Duration	26 June 2026 – 2 July 2026
Primary Domain	Artificial Intelligence, Natural Language Processing, EdTech
Core Technologies	Whisper, Sentence-BERT, Librosa, Gemini API, Streamlit
Document Type	Project Planning, Development and Deployment Report

Project Description

The Voice-Based Concept Understanding Analyser (VBCUA) is an AI-powered educational assessment application designed to evaluate how well a student truly understands a concept, rather than simply how fluently they can speak. Conventional oral assessment tools focus almost entirely on pronunciation, fluency, and grammar, leaving a critical gap: a student may speak smoothly while demonstrating little grasp of the underlying concept, or may understand a topic deeply while struggling to articulate it fluently. VBCUA closes this gap by combining automatic speech recognition, semantic similarity analysis, and acoustic feature extraction into a single, unified understanding-assessment pipeline.

In the VBCUA workflow, a student uploads or records a spoken explanation of a given topic through a Streamlit-based web interface. The recorded audio is transcribed into text using OpenAI's Whisper speech-to-text model. The resulting transcript is then compared against a reference or expected answer using Sentence-BERT sentence embeddings, producing a semantic similarity score that reflects conceptual accuracy rather than exact keyword matching. In parallel, Librosa is used to extract acoustic and prosodic features from the audio signal — including pitch, pace, energy, and pause patterns — which are used to compute a speech-quality score. These two scores are combined into a composite Understanding Score, and the Google Gemini API is used to generate personalised, human-readable feedback that highlights conceptual gaps and suggests improvements. All results, including waveform visualisations, are displayed on an interactive Streamlit dashboard, and the complete assessment can be exported as a downloadable PDF report using ReportLab.

VBCUA is intended for use by students preparing for oral examinations and viva-voce assessments, by educators who wish to conduct scalable formative assessment of conceptual understanding, and by self-learners who want objective, data-driven feedback on how well they can explain what they have learned.

Usage Scenarios

Scenario 1 (Self-Assessment by a Student): A computer science student preparing for a viva records a two-minute spoken explanation of "Object-Oriented Programming" using the microphone recorder in the Streamlit interface. VBCUA transcribes the response, compares it against the reference answer for the topic, and reports an understanding score of 78%, noting that the concepts of encapsulation and polymorphism were explained clearly but inheritance was only briefly mentioned. The AI feedback module suggests the student revisit inheritance with a concrete example before the exam.

Scenario 2 (Classroom Formative Assessment): An instructor uploads twenty pre-recorded student explanations of "Newton's Laws of Motion" collected during a flipped-classroom activity. For each submission, VBCUA generates an understanding score, a speech-quality score, and a short AI-generated feedback summary, allowing the instructor to quickly identify which students need additional support without having to listen to and manually grade every recording.

Scenario 3 (Fluency and Confidence Coaching): A learner preparing for an English-medium technical interview records an explanation of "REST APIs." Beyond the understanding score, VBCUA's speech-feature analysis reports on speaking pace, filler-word frequency, and pause distribution, and the PDF report includes a waveform visualisation alongside targeted suggestions for reducing hesitation and improving pacing.

1. Introduction

Assessing whether a learner has genuinely understood a concept is one of the most persistent challenges in education. Written tests can be gamed through memorisation, multiple-choice questions rarely reveal the depth of reasoning behind an answer, and oral assessments, while effective, are time-consuming to conduct and grade at scale. At the same time, verbal explanation is widely recognised as one of the strongest indicators of conceptual mastery: the ability to explain an idea clearly, in one's own words, using coherent reasoning, generally requires a deeper level of understanding than the ability to recognise a correct option in a list.

The Voice-Based Concept Understanding Analyser (VBCUA) was conceived to bring this style of assessment into a scalable, technology-assisted format. Rather than replacing human evaluators, VBCUA acts as an intelligent first-pass assessment and feedback tool: it converts spoken answers into text, measures how semantically close the answer is to an expected explanation, evaluates delivery quality through acoustic analysis, and produces constructive, AI-generated feedback almost instantly. This allows students to practise explaining concepts as often as they like and receive immediate, objective feedback, and allows educators to triage large volumes of spoken responses efficiently.

This document describes the complete software engineering process followed in building VBCUA over the project duration of 26 June 2026 to 2 July 2026, including requirement analysis, system architecture, module-wise implementation, testing, and deployment. It is intended to serve as a formal project submission that captures both the technical design decisions and the development milestones achieved by Team SWTID-2026-9097.

2. Problem Statement

Existing tools for evaluating spoken responses in an academic setting fall into two broad, and largely disconnected, categories. The first category consists of speech-fluency and pronunciation-assessment tools, commonly used in language-learning applications, which score a speaker on articulation, accent, and grammatical correctness but have no mechanism for judging whether the content of the speech is conceptually correct. The second category consists of text-based automated grading systems, which can compare a written answer against a reference answer using techniques such as keyword matching or semantic similarity, but these systems require the learner's response to already be in text form and therefore cannot be used directly with spoken answers.

This leaves a clear gap: there is no lightweight, accessible system that takes a spoken explanation as direct input and evaluates it simultaneously on (a) how conceptually accurate and complete the explanation is, and (b) how well it was delivered in terms of speech quality, while also providing constructive feedback rather than a bare numeric score. Manually addressing this gap by having instructors listen to every spoken submission does not scale beyond small class sizes, and does not give students an avenue for repeated self-practice outside class hours.

2.1 Specific Problems Identified

- Verbal explanations are rarely assessed for conceptual correctness; existing tools measure fluency, not understanding.
- Manual grading of spoken answers is time-consuming and does not scale to large cohorts or frequent practice.
- Students preparing for viva-voce examinations, interviews, or oral presentations lack an objective, repeatable way to self-test their explanations.
- Feedback on spoken academic answers, when given at all, is often subjective, inconsistent between evaluators, and delayed.
- There is no unified, low-cost application that combines speech-to-text, semantic comparison, acoustic analysis, and AI-generated feedback into a single accessible workflow.

3. Objectives

The VBCUA project was undertaken with the following primary and secondary objectives, which collectively guided the requirement analysis, design, and milestone planning described in later sections of this document.

3.1 Primary Objectives

- To design and develop a web-based application that accepts spoken (recorded or uploaded) explanations of academic concepts as input.
- To accurately transcribe spoken answers into text using a robust, open-source automatic speech recognition model (OpenAI Whisper).
- To quantitatively measure the conceptual similarity between a student's transcribed explanation and an expected reference answer using Sentence-BERT embeddings and cosine similarity.
- To extract and analyse relevant acoustic and prosodic features of the recorded speech (pitch, energy, tempo, pauses) using Librosa, in order to derive an independent speech-quality score.
- To compute a composite Understanding Score that reflects both conceptual accuracy and delivery quality, along with sub-scores that can be inspected independently.
- To generate natural-language, constructive feedback using the Google Gemini API that highlights strengths, gaps, and improvement suggestions specific to the student's response.

3.2 Secondary Objectives

- To provide an intuitive, single-page Streamlit dashboard that visualises transcripts, scores, and audio waveforms in an easy-to-interpret layout.
- To allow users to export a complete, professionally formatted PDF report of their assessment session using ReportLab, suitable for sharing with instructors or for personal record-keeping.
- To design the system with a modular architecture so that individual components (speech-to-text, semantic scoring, feature extraction, feedback generation, reporting) can be independently tested, replaced, or extended.
- To optionally persist assessment history in a lightweight SQLite database so that a learner's progress can be tracked across multiple sessions.
- To ensure the application can be developed, tested, and demonstrated within the constrained one-week project timeline using freely available models and APIs.

4. System Architecture

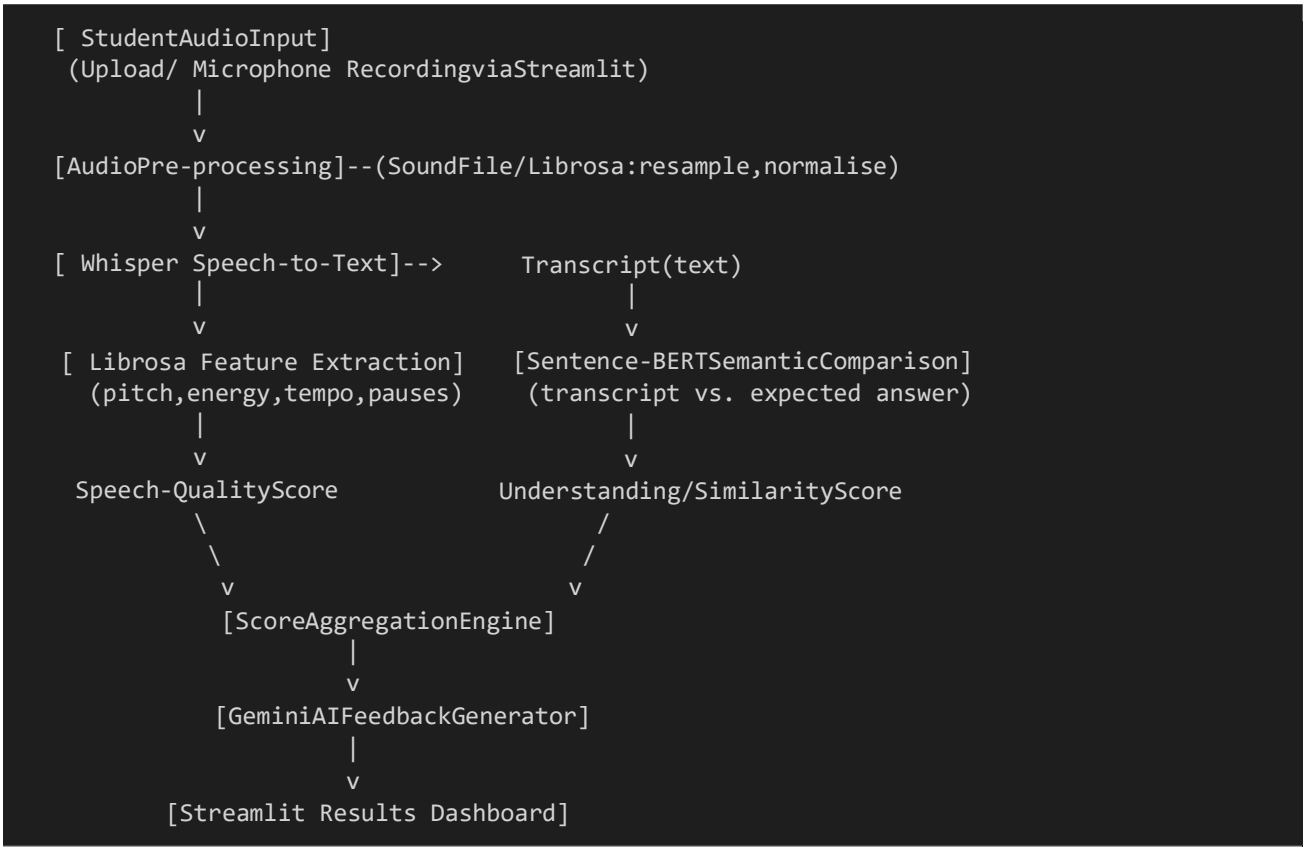
VBCUA follows a modular, layered architecture that separates presentation, orchestration, and AI/analysis concerns. This separation keeps each functional unit independently testable and allows individual models (for example, the speech-to-text engine or the feedback-generation model) to be upgraded without requiring changes to the rest of the system.

4.1 Architectural Layers

- **Presentation Layer:** A Streamlit application that renders the topic-selection interface, the audio upload/recording widget, the results dashboard (scores, transcript, waveform), and the PDF download control.
- **Orchestration Layer:** A Python backend module that receives the uploaded audio, coordinates calls to the speech-to-text, semantic-analysis, audio-feature, and feedback-generation modules in sequence, and assembles their outputs into a single result object consumed by the dashboard.
- **AI / Analysis Layer:** Four independent processing components — the Whisper transcription engine, the Sentence-BERT semantic similarity engine, the Librosa acoustic feature extractor, and the Gemini feedback generator — each exposed through a simple function-level interface.
- **Persistence Layer (optional):** A lightweight SQLite database used to store topic definitions, reference answers, and historical assessment records for progress tracking.
- **Reporting Layer:** A ReportLab-based PDF generator that compiles the transcript, scores, waveform image, and AI feedback into a downloadable report.

4.2 High-Level Data Flow

The diagram below represents the logical flow of data through the VBCUA pipeline, from audio input to the final downloadable report.



```
        (scores,transcript,waveformchart)
                        |
                        v
          [ReportLabPDFReportExport]
```

4.3 Component Interaction Description

When a user submits an audio file, the orchestration layer first validates the file format and duration, then invokes the audio pre-processing routine to standardise the sample rate and channel count using Librosa and SoundFile. The cleaned audio is passed to Whisper, which returns a plain-text transcript. This transcript is forwarded to the Sentence-BERT module, which encodes both the transcript and the expected reference answer into fixed-length embeddings and computes their cosine similarity to produce an Understanding Score between 0 and 100. Independently, the original audio signal is analysed by the Librosa-based feature extractor to compute pitch variation, articulation rate, energy levels, and pause ratios, which are normalised into a Speech-Quality Score. Both scores, together with the transcript and topic metadata, are sent to the Gemini API with a structured prompt requesting concise, constructive feedback. Finally, all outputs are rendered on the Streamlit dashboard and made available for PDF export.

5. Technology Stack

VBCUA was built entirely using open-source and freely accessible tools appropriate for a one-week academic project, while still reflecting the technology choices that would be used in a production-grade conceptual-assessment system.

Layer / Purpose	Technology	Role in VBCUA
Frontend / Dashboard	Streamlit	Renders the upload/recording UI, results dashboard, waveform charts, and download controls.
Backend Logic	Python 3.10+	Coordinates the processing pipeline and connects all modules.
Speech-to-Text	OpenAI Whisper	Converts spoken audio explanations into text transcripts.
Semantic Analysis	Sentence-BERT (Sentence-Transformers)	Encodes transcript and reference answer for semantic similarity scoring.
AI Feedback Generation	Google Gemini API	Generates natural-language, personalised feedback on the student's explanation.
Audio Processing	Librosa, SoundFile	Extracts pitch, tempo, energy, and pause features; handles audio I/O and format conversion.
Data Processing	NumPy, Pandas	Numerical computation, score aggregation, and tabular history management.
Visualisation	Matplotlib	Generates waveform and feature visualisations for the dashboard and PDF report.
PDF Report Generation	ReportLab	Compiles transcript, scores, and charts into a downloadable PDF report.
Machine Learning Framework	PyTorch	Underlying tensor framework for Whisper and Sentence-BERT models.
Database (optional)	SQLite	Stores topics, reference answers, and historical assessment records.
IDE	Visual Studio Code	Primary development environment used by the team.
Version Control	Git & GitHub	Source control, collaboration, and milestone tracking.

6. Pre-requisites

Before development began, the team ensured familiarity with the following tools, libraries, and concepts, which formed the technical foundation for the project.

- **Python Programming Proficiency:** Comfort with Python 3, virtual environments, and package management using pip.
- **Streamlit Framework Knowledge:** Understanding of Streamlit's script-execution model, session state, widgets, and layout containers.
- **Speech Processing Fundamentals:** Basic understanding of audio sampling rates, waveform representation, and automatic speech recognition concepts required to work with Whisper.
- **NLP and Embeddings:** Familiarity with sentence embeddings, cosine similarity, and the Sentence-Transformers library.
- **Generative AI API Usage:** Ability to construct prompts and parse structured responses from the Google Gemini API.
- **Audio Feature Engineering:** Working knowledge of Librosa for extracting pitch (F0), RMS energy, tempo, and silence/pause detection.
- **Version Control with Git:** Ability to manage branches, commits, and pull requests on GitHub for collaborative development.
- **Development Environment Setup:** Experience configuring Python virtual environments and installing GPU/CPU-appropriate PyTorch builds.

7. Project Workflow

Given the one-week project duration, the development effort was organised into five sequential milestones, each building on the deliverables of the previous one. The table below summarises the milestone schedule before each milestone is expanded in detail in the following sections.

Milestone	Focus Area	Planned Duration
Milestone 1	Project Planning & Requirement Analysis	26 – 27 June 2026
Milestone 2	Core Functionalities Development	27 – 28 June 2026
Milestone 3	Backend & AI Development	28 – 30 June 2026
Milestone 4	Frontend Development	30 June – 1 July 2026
Milestone 5	Testing & Deployment	1 – 2 July 2026

Milestone 1: Project Planning & Requirement Analysis

- Activity 1.1: Define project scope, target users, and success criteria.
- Activity 1.2: Research and select appropriate models for speech-to-text, semantic similarity, and feedback generation.
- Activity 1.3: Design the system architecture and data flow between modules.
- Activity 1.4: Set up the development environment, repository structure, and dependencies.

Milestone 2: Core Functionalities Development

- Activity 2.1: Implement audio ingestion, validation, and pre-processing.
- Activity 2.2: Build the topic and reference-answer management module.

Milestone 3: Backend & AI Development

- Activity 3.1: Integrate Whisper for speech-to-text transcription.
- Activity 3.2: Integrate Sentence-BERT for semantic similarity scoring.
- Activity 3.3: Implement Librosa-based acoustic feature extraction and speech-quality scoring.
- Activity 3.4: Integrate the Gemini API for AI-generated feedback and implement the score-aggregation engine.

Milestone 4: Frontend Development

- Activity 4.1: Design and build the Streamlit input interface (topic selection, upload/recording).
- Activity 4.2: Build the results dashboard, including waveform visualisation and score displays.
- Activity 4.3: Implement the PDF report generation and download feature.

Milestone 5: Testing & Deployment

- Activity 5.1: Conduct unit, integration, and user-acceptance testing across all modules.
- Activity 5.2: Optimise performance and handle edge cases (silence, noisy audio, short answers).
- Activity 5.3: Deploy the application locally and prepare it for public/cloud deployment.

Milestone 1: Project Planning & Requirement Analysis

This milestone establishes the foundation of the VBCUA project. The team focused on clearly defining what the application must accomplish, evaluating candidate AI models for each processing stage, laying out the system architecture, and preparing a working development environment before any feature code was written.

Activity 1.1: Define Project Scope, Users, and Success Criteria

- Identify the primary users of the system: individual students practising oral explanations, and educators conducting formative assessment of spoken answers.
- Define the core capability boundary: VBCUA evaluates conceptual correctness and speech delivery quality from a single spoken response against one expected reference answer per topic; it does not perform open-ended conversational tutoring.
- Establish measurable success criteria: transcription accuracy sufficient for reliable semantic comparison, an understanding score that meaningfully differentiates strong and weak explanations, and end-to-end processing time suitable for interactive use (under roughly 30 seconds for a two-minute recording).
- Document assumptions and constraints, including the one-week development timeline, reliance on free-tier or open-source models, and the requirement that the application run on a standard laptop without dedicated GPU hardware.

Activity 1.2: Research and Select Appropriate AI Models

Choosing the right model for each pipeline stage was critical, given the tight project timeline and the need for the models to run reliably on commodity hardware. The team evaluated the following options:

- **Speech-to-Text:** Compared OpenAI Whisper (tiny, base, and small checkpoints) against cloud-only alternatives. Whisper was selected because it runs locally, supports multiple accents, and the "base" model offers an acceptable balance between transcription accuracy and inference speed on CPU.
- **Semantic Similarity:** Evaluated raw keyword-overlap methods (TF-IDF, Jaccard similarity) against embedding-based approaches. Sentence-BERT (specifically the all-MiniLM-L6-v2 checkpoint) was selected because sentence embeddings capture paraphrased and reworded explanations far better than keyword matching, which is essential since students rarely use the exact wording of a reference answer.
- **AI Feedback Generation:** Considered several large language model APIs and selected Google Gemini for its strong instruction-following ability, generous free-tier quota suitable for the project timeline, and fast response times when generating short, structured feedback.
- **Audio Feature Extraction:** Selected Librosa for its comprehensive, well-documented set of functions for pitch tracking, energy computation, tempo estimation, and silence detection, all of which are needed to compute the speech-quality sub-score.

Activity 1.3: Define the System Architecture

- Draft an architectural diagram covering the presentation layer (Streamlit), orchestration layer (Python pipeline controller), AI/analysis layer (Whisper, Sentence-BERT, Librosa, Gemini), and reporting layer (ReportLab).
- Define the data contract passed between modules: an audio file object flows into the pipeline, and a structured result dictionary (transcript, understanding_score, speech_score, overall_score, feedback_text, waveform_image) flows out to the dashboard.

- Decide that the reference/expected answers for each topic will be stored as structured text (initially in a Python dictionary or JSON file, with optional SQLite persistence) so that new topics can be added without code changes.
- Specify error-handling responsibilities at each layer — for example, the orchestration layer must gracefully handle empty transcripts, unsupported audio formats, and API timeouts from Gemini.

Activity 1.4: Set Up the Development Environment

The team prepared a consistent local development environment to avoid dependency conflicts between the several machine-learning libraries used in the project.

```
C:\vbcua> python -m venv venv
C:\vbcua>venv\Scripts\activate
(venv)C:\vbcua>pipinstallstreamlitopenai-whispersentence-transformers (venv)
C:\vbcua> pip install librosa soundfile numpy pandas matplotlib
(venv)C:\vbcua> pipinstallreportlabgoogle-generativeaipython-dotenvtorch
```

- Create a .env file to securely store the GEMINI_API_KEY, following the same pattern used for API-key management in similar generative-AI projects, and add .env to .gitignore.
- Initialise a Git repository and push the initial project skeleton to GitHub, establishing a branching convention (main, dev, and feature branches per milestone) for the remainder of the project.
- Set up the base folder structure (app.py, modules/, assets/, data/, reports/) so subsequent milestones could add code into a predictable layout, described in full in Section 12 of this document.

Milestone 2: Core Functionalities Development

With the architecture and environment in place, Milestone 2 focused on building the foundational, non-AI plumbing of the application: audio ingestion and validation, topic and reference-answer management, and the skeleton of the Streamlit application into which the AI modules would later be plugged.

Activity 2.1: Implement Audio Ingestion and Pre-processing

- **File Upload Support:** Enable users to upload pre-recorded audio files in common formats (WAV, MP3, M4A) through Streamlit's file_uploader widget.
- **Live Microphone Recording:** Provide an in-browser recording widget so users can record their explanation directly, without needing a separate recording application.
- **Format Validation:** Check file size, duration, and sample rate on upload, rejecting empty or excessively long recordings with a clear error message before any processing begins.
- **Standardisation:** Use SoundFile and Librosa to resample all incoming audio to a consistent 16 kHz mono format, which is the input format expected by Whisper and by the acoustic-feature extraction routines.

Activity 2.2: Build the Topic and Reference-Answer Management Module

- Design a simple data model for a "Topic", consisting of a topic name, subject area, an expected reference answer (used for semantic comparison), and an optional difficulty level.
- Populate an initial set of sample topics (for example, Object-Oriented Programming, Newton's Laws of Motion, TCP/IP Networking Basics, Photosynthesis) with concise reference answers for use during development and testing.
- Implement functions to load topics from a JSON file or SQLite table, and to add new topics through a simple administrative form, so that the topic bank is extensible without modifying application code.
- Build the initial Streamlit page skeleton with a topic-selection dropdown, a placeholder area for the audio input widget, and a placeholder results section, establishing the page layout that Milestone 4 would later complete.

Milestone 3: Backend & AI Development

Milestone 3 is the technical core of the VBCUA project. It covers the integration of all four AI/analysis components — speech-to-text transcription, semantic similarity scoring, acoustic feature extraction, and AI feedback generation — as well as the score-aggregation logic that combines their outputs into a single, interpretable Understanding Score.

Activity 3.1: Integrate Whisper for Speech-to-Text Transcription

The transcription module wraps OpenAI's Whisper model behind a simple function interface that the orchestration layer can call with a pre-processed audio file path.

```
importwhisper

_model=whisper.load_model("base")

deftranscribe_audio(audio_path:str)->str:
    """Transcribespeechinthegivenaudiofiletotextusing Whisper."""  result =
    _model.transcribe(audio_path, language="en", fp16=False) return
    result["text"].strip()
```

- Load the Whisper "base" checkpoint once at application startup and cache it in memory (using Streamlit's @st.cache_resource) to avoid reloading the model on every user interaction.
- Force language="en" for the initial release to keep transcription latency low, with multi-language support noted as a future enhancement.
- Handle silent or near-silent recordings by detecting empty transcripts and returning a user-facing message prompting the student to re-record their answer.

Activity 3.2: Integrate Sentence-BERT for Semantic Similarity Scoring

The semantic-analysis module encodes both the transcript and the topic's expected reference answer into embeddings using a pretrained Sentence-BERT model, then computes cosine similarity between them.

```
from sentence_transformersimport SentenceTransformer,util

_embedder= SentenceTransformer("all-MiniLM-L6-v2")

defcompute_understanding_score(transcript:str,expected_answer:str)->float:
    """Return a 0-100 understanding score from semantic similarity."""
    embeddings=_embedder.encode([transcript,expected_answer],convert_to_tensor=True)
    similarity = util.cos_sim(embeddings[0], embeddings[1]).item()
    returnround(max(similarity,0)*100,2)
```

- Select the all-MiniLM-L6-v2 checkpoint for its strong trade-off between embedding quality and inference speed, making it suitable for near-real-time scoring on CPU.
- Normalise the raw cosine similarity value (typically ranging from -1 to 1) into a 0–100 scale for presentation to students.
- Additionally compute keyword-level coverage (which key terms from the reference answer appear in the transcript) as a supporting diagnostic used in the AI feedback prompt, without altering the primary semantic score.

Activity 3.3: Implement Librosa-Based Acoustic Feature Extraction

The speech-quality module analyses the raw audio waveform independently of its transcribed content, focusing purely on delivery characteristics.

```
import librosa
importnumpyasnp

defextract_speech_features(audio_path:str)->dict: y,
    sr = librosa.load(audio_path, sr=16000) pitches,
    _ = librosa.piptrack(y=y, sr=sr)
    tempo,_=librosa.beat.beat_track(y=y,sr=sr) rms
    = librosa.feature.rms(y=y)
    intervals= librosa.effects.split(y,top_db=25)
    pause_ratio=1 -(sum(e-sfors,einintervals)/len(y)) return {
        "pitch_std":float(np.std(pitches[pitches>0]))ifnp.any(pitches>0)else0.0, "tempo":
        float(tempo),
        "energy": float(np.mean(rms)),
        "pause_ratio":float(pause_ratio),
    }

defcompute_speech_quality_score(features:dict)->float:
    """Combineacousticfeaturesintoasingle0-100speech-qualityscore."""
    tempo_score = 100 - min(abs(features["tempo"] - 110), 50) * 2
    pause_score=max(0,100-features["pause_ratio"]*150) energy_score =
    min(features["energy"] * 4000, 100)
    returnround((tempo_score+pause_score+energy_score)/3,2)
```

- Extract pitch contour statistics using piptrack to gauge intonation variability, which relates to expressiveness and confidence.
- Estimate speaking tempo using beat_track as a proxy for words-per-minute pacing, scoring recordings closer to a natural conversational pace more favourably.
- Compute the pause ratio using librosa.effects.split, which detects non-silent intervals, so that excessive hesitation or long pauses reduce the speech-quality score.
- Combine the individual acoustic sub-scores into a single normalised Speech-Quality Score reported alongside the Understanding Score.

Activity 3.4: Integrate Gemini API for Feedback and Build the Score-Aggregation Engine

The final backend activity ties together the outputs of the previous three modules and produces the feedback students actually read.

```
importgoogle.generativeaiasgenai

genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
_gemini_model=genai.GenerativeModel("gemini-1.5-flash")

defgenerate_feedback(topic:str,transcript: str,expected_answer:str,
                     understanding_score: float, speech_score: float) -> str:
    prompt= f"""You are anexpertacademicevaluator. Astudentwasaskedtoexplain the
    topic '{topic}'. Expected answer: {expected_answer}
    Student'stranscribed explanation:{transcript}
    Understandingscore:{understanding_score}/100.Speechqualityscore:
{speech_score}/100.
    Giveconcise,constructivefeedback(under120words)covering:whatthestudentgot
    right,whichconceptsweremissedorunclear,andonespecificsuggestiontoimprove
```

```
    bothunderstandinganddelivery."""
    response=_gemini_model.generate_content(prompt)
    return response.text.strip()

defcompute_overall_score(understanding_score:float,speech_score:float)->float:
    """Weighted aggregation favouring conceptual understanding over delivery."""
    return round(understanding_score * 0.7 + speech_score * 0.3, 2)
```

- Construct a structured prompt that supplies Gemini with the topic, the expected answer, the transcript, and both numeric sub-scores, so the generated feedback is grounded in the actual assessment data rather than generic commentary.
- Weight the overall score 70% toward conceptual Understanding and 30% toward Speech Quality, reflecting the project's core premise that conceptual correctness matters more than delivery polish, while still rewarding clear articulation.
- Wrap all Gemini API calls in try/except blocks with retry logic and a fallback rule-based feedback message, ensuring the dashboard still displays useful information if the API is temporarily unavailable.
- Assemble the complete pipeline into a single orchestration function, analyse_response(), that sequentially calls transcription, semantic scoring, feature extraction, score aggregation, and feedback generation, returning one consolidated result dictionary to the frontend.

Milestone 4: Frontend Development

Milestone 4 focused on transforming the backend pipeline into an accessible, visually clear Streamlit dashboard. The emphasis was on making the input flow effortless for students and presenting the resulting scores, transcript, and feedback in a layout that is easy to scan at a glance.

Activity 4.1: Design and Build the Input Interface

- Build the topic-selection section using a Streamlit selectbox populated from the topic-management module built in Milestone 2, displaying the associated subject area as supporting context.
- Add a tabbed input section offering two ways to provide a spoken answer: an st.file_uploader for pre-recorded files, and an embedded audio-recorder component for live microphone capture.
- Display a live character/time counter and basic recording-quality hints (for example, a warning if the recording is shorter than 10 seconds) before the user submits their response.
- Add an "Analyse My Explanation" action button that triggers the backend pipeline, disabling itself and showing a Streamlit spinner while transcription and scoring are in progress.

Activity 4.2: Build the Results Dashboard

- **Score Panel:** Three prominent metric cards (using st.metric) displaying the Understanding Score, Speech-Quality Score, and Overall Score, each with a colour-coded delta indicating performance band (needs improvement, good, excellent).
- **Transcript Panel:** A collapsible panel showing the Whisper-generated transcript alongside the topic's expected reference answer for direct comparison.
- **Waveform Visualisation:** A Matplotlib waveform plot of the submitted audio, rendered inline via st.pyplot, with shaded regions marking detected pauses to make the pacing feedback tangible.
- **AI Feedback Panel:** A styled text panel presenting the Gemini-generated feedback in a readable, paragraph format, separated visually from the numeric scores.
- Implement renderResults(result) style logic in the Streamlit script to update all of the above panels reactively once the analysis pipeline returns its output, using Streamlit's session state to avoid recomputation on unrelated widget interactions.

Activity 4.3: Implement PDF Report Generation and Download

```
from reportlab.lib.pagesizesimportA4
fromreportlab.platypusimportSimpleDocTemplate,Paragraph,Spacer,Image from
reportlab.lib.styles import getSampleStyleSheet

def build_pdf_report(output_path: str,topic: str, transcript:str,
                     scores:dict,feedback:str,waveform_image_path:str): doc =
    SimpleDocTemplate(output_path, pagesize=A4)
    styles=getSampleStyleSheet()
    story = [
        Paragraph("VBCUA Assessment Report", styles["Title"]),
        Paragraph(f"Topic: {topic}", styles["Heading2"]),
        Paragraph(f"UnderstandingScore:{scores['understanding']}/100",
styles["Normal"]),
        Paragraph(f"SpeechQualityScore:{scores['speech']}/100",styles["Normal"]),
        Paragraph(f"Overall Score: {scores['overall']} / 100", styles["Normal"]),
        Spacer(1, 12),
        Paragraph("Transcript",styles["Heading2"]),
```

```
        Paragraph(transcript,styles["Normal"]),
        Spacer(1, 12),
        Paragraph("AIFeedback",styles["Heading2"]),
        Paragraph(feedback, styles["Normal"]),
        Spacer(1, 12),
        Image(waveform_image_path,width=400,height=150),
    ]
    doc.build(story)
```

- Generate the waveform image to a temporary PNG file using Matplotlib so it can be embedded directly into the PDF via ReportLab's Image flowable.
- Provide an st.download_button on the dashboard that serves the generated PDF file to the user's browser without requiring server-side file management beyond a temporary reports/ directory.
- Ensure the PDF layout mirrors the on-screen dashboard structure (scores, transcript, feedback, waveform) so that the exported report is self-contained and does not require the live application to be understood.

Milestone 5: Testing & Deployment

The final milestone verified that VBCUA behaves correctly and reliably across a representative range of inputs, and prepared the application for local and shareable deployment within the remaining project days.

Activity 5.1: Conduct Testing Across All Modules

- Unit-test the semantic-scoring function with known input pairs (identical text, paraphrased text, and unrelated text) to confirm the similarity score behaves monotonically as expected.
- Unit-test the acoustic-feature extractor against short synthetic and recorded clips with known characteristics (fast speech, slow speech, silence-heavy recordings) to validate the tempo and pause-ratio calculations.
- Integration-test the full analyse_response() pipeline end-to-end using sample recordings for each topic in the topic bank, confirming transcripts, scores, and feedback are produced without runtime errors.
- Conduct informal user-acceptance testing with a small group of student volunteers, collecting feedback on interface clarity, scoring intuitiveness, and perceived usefulness of the AI feedback.

Activity 5.2: Optimise Performance and Handle Edge Cases

- Add explicit handling for empty or near-silent recordings, returning a clear "No speech detected — please try recording again" message instead of allowing the pipeline to fail downstream.
- Add input-length guards, capping recordings at a configurable maximum duration (for example, 5 minutes) to keep Whisper transcription time bounded.
- Cache the Whisper and Sentence-BERT models using Streamlit's resource caching so that repeated analyses within a session do not reload multi-hundred-megabyte models from disk each time.
- Add graceful degradation for Gemini API failures or rate limits, falling back to a template-based feedback message derived directly from the numeric scores.

Activity 5.3: Deployment

VBCUA was deployed first locally to validate correct operation end-to-end, and then made available for demonstration through Streamlit's sharing mechanism.

```
(venv)C:\vbcua>streamlitrunapp.py

  YoucannowviewyourStreamlitappinyourbrowser. Local
  URL:          http://localhost:8501
  NetworkURL:http://192.168.1.24:8501
```

- Run the application locally with streamlit run app.py and verify all five processing stages (upload/record, transcription, scoring, feedback, PDF export) using each sample topic.
- Freeze exact dependency versions into a requirements.txt file to ensure reproducible installation on any evaluation machine.
- Prepare the application for optional cloud deployment via Streamlit Community Cloud, adding the GEMINI_API_KEY as a secret in the deployment environment rather than committing it to source control.
- Document known operational limits (approximate per-request processing time, maximum recording length, and API quota considerations) for anyone deploying or evaluating the application.

8. Module-Wise Explanation

VBCUA is organised into clearly separated Python modules, each responsible for a single stage of the assessment pipeline. This section describes the purpose and responsibilities of each module.

8.1 audio_input.py — Audio Ingestion Module

Handles receiving audio from either the file-upload widget or the microphone recorder, validating its format and duration, and standardising it (mono channel, 16 kHz sample rate) before it is passed further down the pipeline. This module isolates all audio I/O concerns so that downstream modules can assume a consistent, clean input format.

8.2 transcription.py — Speech-to-Text Module

Wraps the Whisper model and exposes a single transcribe_audio() function. Responsible for loading and caching the model, invoking transcription, and returning cleaned transcript text. Also detects and flags empty or low-confidence transcriptions.

8.3 semantic_analysis.py — Understanding Scoring Module

Wraps the Sentence-BERT model and computes the semantic similarity between a transcript and the topic's expected answer, converting the resulting cosine similarity into a 0–100 Understanding Score. It also performs a supporting keyword-coverage check used to enrich the AI feedback prompt.

8.4 speech_features.py — Acoustic Analysis Module

Uses Librosa to extract pitch, tempo, energy, and pause-ratio features from the raw audio waveform, and aggregates them into a single Speech-Quality Score independent of the spoken content.

8.5 feedback_engine.py — AI Feedback Module

Constructs a structured prompt from the topic, transcript, expected answer, and computed scores, sends it to the Gemini API, and returns concise, human-readable feedback. Includes a rule-based fallback for API failures.

8.6 scoring.py — Score Aggregation Module

Combines the Understanding Score and Speech-Quality Score into a single weighted Overall Score, and defines the performance bands (Needs Improvement, Good, Excellent) used for colour-coding on the dashboard.

8.7 report_generator.py — PDF Reporting Module

Uses ReportLab to assemble the transcript, scores, waveform chart, and AI feedback into a single downloadable PDF report, mirroring the layout of the on-screen dashboard.

8.8 topics.py — Topic and Reference-Answer Management Module

Defines the data model for a topic (name, subject, expected answer, difficulty) and provides functions to load, add, and retrieve topics, either from a JSON file or an optional SQLite database.

8.9 app.py — Application Entry Point

The main Streamlit script that lays out the page, wires together all of the modules above into the analyse_response() pipeline, manages session state, and renders the results dashboard.

9. Folder Structure

The project follows a modular folder structure that keeps AI/processing logic, static assets, sample data, and generated reports clearly separated from the main application entry point.

```
vbcua/
├──app.py                        #Streamlitapplicationentrypoint
├──requirements.txt               # PinnedPythondependencies
├──.env                           #GEMINI_API_KEY(excludedfrom versioncontrol)
├──.gitignore
├──modules/
│    ├──audio_input.py            #Audioingestion&pre-processing
│    ├──transcription.py          # Whisperspeech-to-textwrapper
│    ├──semantic_analysis.py      #Sentence-BERTunderstandingscoring
│    ├──speech_features.py        #Librosa acousticfeatureextraction
│    ├──feedback_engine.py        # GeminiAPIfeedbackgeneration
│    ├──scoring.py                 #Scoreaggregationlogic
│    ├──report_generator.py       #ReportLabPDFgeneration
│    └──topics.py                  #Topicandreference-answermanagement
├──data/
│    ├──topics.json                #Sampletopicsandreferenceanswers
│    └──vbcua.db                   #OptionalSQLitehistorydatabase
├──assets/
│    ├──logo.png
│    └──sample_audio/              #Samplerecordingsusedfortesting
├──reports/                        #Temporarygenerated PDFreports
└──tests/
     ├──test_semantic_analysis.py
     ├──test_speech_features.py
     └──test_pipeline.py
```

10. Workflow Explanation

This section walks through the complete end-to-end workflow a user experiences when using VBCUA, tying together the modules described above into a single narrative.

- **Step 1 — Topic Selection:** The user selects a topic from the dropdown populated by topics.py; the corresponding expected answer is loaded into session state.
- **Step 2 — Audio Submission:** The user either uploads a recording or records live audio through the microphone widget; audio_input.py validates and standardises the file.
- **Step 3 — Transcription:** transcription.py invokes Whisper to convert the audio into a text transcript.
- **Step 4 — Parallel Analysis:** semantic_analysis.py compares the transcript to the expected answer to compute the Understanding Score, while speech_features.py analyses the raw audio to compute the Speech-Quality Score.
- **Step 5 — Score Aggregation:** scoring.py combines both sub-scores into a single weighted Overall Score and assigns a performance band.
- **Step 6 — Feedback Generation:** feedback_engine.py sends the topic, transcript, expected answer, and scores to the Gemini API and receives personalised feedback text.
- **Step 7 — Dashboard Rendering:** app.py updates the Streamlit dashboard with the scores, transcript, waveform chart, and feedback.
- **Step 8 — Report Export:** On request, report_generator.py compiles all of the above into a downloadable PDF via the dashboard's download button.

11. Important Python Functions / Modules

The following table summarises the most important functions in the VBCUA codebase, describing their purpose and behaviour without reproducing the complete implementation, in keeping with a design-level project overview.

Function	Module	Description
transcribe_audio(audio_path)	transcription.py	Loads the cached Whisper model and returns the transcribed text for a given pre-processed audio file.
compute_understanding_score(transcript, expected)	semantic_analysis.py	Encodes both texts with Sentence-BERT and returns a 0–100 score based on their cosine similarity.
extract_speech_features(audio_path)	speech_features.py	Uses Librosa to compute pitch variability, tempo, RMS energy, and pause ratio for the given audio.
compute_speech_quality_score(features)	speech_features.py	Normalises and combines the extracted acoustic features into a single 0–100 delivery-quality score.
generate_feedback(topic, transcript, expected, u_score, s_score)	feedback_engine.py	Builds a structured prompt and calls the Gemini API to produce concise, constructive feedback text.
compute_overall_score(u_score, s_score)	scoring.py	Applies a 70/30 weighted average to combine Understanding and Speech-Quality scores into an Overall Score.
build_pdf_report(...)	report_generator.py	Assembles the transcript, scores, waveform image, and feedback into a formatted, downloadable PDF file.
load_topics() / get_topic(name)	topics.py	Reads the topic bank from JSON/SQLite and retrieves a specific topic's expected reference answer.
analyse_response(audio_path, topic)	app.py (orchestrator)	Coordinates the full pipeline end-to-end, calling each module in sequence and returning a consolidated results dictionary.

12. Testing Process

Testing for VBCUA followed a layered strategy covering isolated unit tests for numerical and model-driven functions, integration tests across the full pipeline, and informal usability testing with real users, given the constrained project timeline.

12.1 Unit Testing

- Verified compute_understanding_score() returns near-100 scores for identical text pairs, high scores for paraphrased pairs, and low scores for unrelated text pairs.
- Verified extract_speech_features() produces sensible tempo and pause-ratio values on synthetic clips of known characteristics (constant tone, silence-only, rapid speech).
- Verified compute_overall_score() correctly applies the 70/30 weighting and stays within the 0–100 bound for all valid sub-score combinations.

12.2 Integration Testing

- Ran the complete analyse_response() pipeline against a library of sample recordings for every topic in the topic bank, confirming no unhandled exceptions occur across the full transcription-to-feedback chain.
- Verified that PDF reports generated via build_pdf_report() open correctly and visually match the corresponding on-screen dashboard results.
- Tested behaviour under simulated API failure conditions (invalid Gemini key, network timeout) to confirm the fallback feedback message is displayed instead of a crash.

12.3 Edge-Case and Negative Testing

- Submitted a silent audio clip and confirmed the system returns a clear "no speech detected" message rather than a misleading low score.
- Submitted an unrelated explanation (for a completely different topic) and confirmed the Understanding Score is appropriately low.
- Submitted an oversized audio file exceeding the configured duration limit and confirmed the upload is rejected with an explanatory message.

12.4 User Acceptance Testing

- Collected feedback from a small group of student testers on the clarity of the score presentation and the usefulness of the AI-generated feedback.
- Iterated on the wording of dashboard labels and score bands based on tester confusion around the difference between the Understanding and Speech-Quality scores.

13. Deployment Process

Deployment of VBCUA was carried out in two stages: local deployment for development-time verification, followed by preparation for shareable cloud deployment.

13.1 Local Deployment

```
(venv)C:\vbcua>pipinstall-rrequirements.txt (venv)
C:\vbcua> streamlit run app.py

  LocalURL:    http://localhost:8501
```

- Install all pinned dependencies from requirements.txt inside the project's virtual environment.
- Ensure the GEMINI_API_KEY environment variable is set via the .env file, loaded automatically using python-dotenv at application startup.
- Launch the app using streamlit run app.py and validate all functional paths locally before considering the build stable.

13.2 Cloud/Shareable Deployment

- Push the finalised codebase, excluding the .env file, to the project's GitHub repository.
- Deploy the application via Streamlit Community Cloud, linking it to the GitHub repository and specifying app.py as the entry point.
- Configure the GEMINI_API_KEY as an encrypted secret within the deployment platform rather than committing it to source control.
- Verify the deployed instance by repeating the core test scenarios (topic selection, upload, recording, scoring, PDF export) against the live public URL.

13.3 Operational Notes

- Whisper and Sentence-BERT models are loaded once per server instance and cached, so the first request after a deployment restart is noticeably slower than subsequent requests.
- Gemini API usage is subject to free-tier rate limits; the fallback feedback mechanism ensures the application remains usable even if the quota is temporarily exceeded.
- Recording length is capped to keep per-request processing time predictable on CPU-only deployment environments.

14. Application Pages / Screens Explanation

14.1 Home / Topic Selection Screen

Description: The landing view of the single-page Streamlit application. It presents the VBCUA title and tagline, a dropdown for selecting the topic to be explained, and a brief display of the topic's subject area and difficulty level. This screen orients the user before they proceed to record or upload their explanation.

14.2 Audio Input Screen

Description: A tabbed section offering two input modes — "Upload Recording" and "Record Now." The upload tab accepts WAV, MP3, and M4A files through a drag-and-drop widget, while the record tab provides an in-browser microphone recorder with a visible timer. An "Analyse My Explanation" button becomes active once valid audio has been provided.

14.3 Processing / Loading State

Description: While the backend pipeline executes, the interface displays a Streamlit spinner with status text ("Transcribing your explanation…", "Comparing with expected answer…", "Generating feedback…") so the user understands that multiple AI stages are running sequentially.

14.4 Results Dashboard

Description: The primary output screen, organised into a clear two-column layout. The left column displays the three score metrics (Understanding, Speech Quality, Overall) as colour-coded cards, followed by the collapsible transcript-versus-expected-answer comparison panel. The right column displays the Matplotlib waveform chart with shaded pause regions, followed by the AI-generated feedback panel in readable prose.

14.5 Report Download Screen

Description: A dedicated action area beneath the results dashboard offering a "Download PDF Report" button. Selecting it triggers report_generator.py to compile the current session's transcript, scores, waveform, and feedback into a formatted PDF, which the browser then downloads directly.

15. Conclusion

The Voice-Based Concept Understanding Analyser demonstrates how open-source speech recognition, sentence-embedding models, acoustic feature engineering, and generative AI feedback can be combined into a single, coherent application that assesses conceptual understanding directly from spoken explanations, rather than relying solely on fluency or written text. Over the course of the project, from 26 June 2026 to 2 July 2026, the team progressed through requirement analysis and model selection, core functionality development, backend AI integration, an interactive Streamlit frontend, and finally rigorous testing and deployment, resulting in a working end-to-end pipeline that transcribes, scores, and provides personalised feedback on a student's spoken answer within a single interactive session.

The resulting system directly addresses the gap identified in the problem statement: it gives students an accessible, repeatable way to practise explaining concepts aloud and receive immediate, objective feedback on both what they said and how they said it, while giving educators a scalable first-pass tool for reviewing spoken responses. The modular architecture — separating transcription, semantic scoring, acoustic analysis, feedback generation, and reporting into independent components — ensures that any single stage of the pipeline can be upgraded or replaced in future iterations without disrupting the rest of the system.

16. Future Scope

- **Multi-Language Support:** Extend transcription and semantic comparison to additional languages using Whisper's multilingual checkpoints and multilingual Sentence-BERT models.
- **Adaptive Reference Answers:** Allow multiple acceptable reference explanations per topic, using the best-matching reference for scoring rather than a single fixed answer.
- **User Accounts and Progress Tracking:** Introduce authentication and persistent history so learners can track improvement in their Understanding and Speech-Quality scores across multiple practice sessions over time.
- **Instructor Dashboard:** Build a batch-review interface allowing educators to upload and evaluate an entire class's recordings at once, with aggregate analytics on common misconceptions.
- **Real-Time Feedback:** Move from post-recording batch analysis toward streaming transcription and live feedback as the student speaks.
- **Fine-Tuned Domain Models:** Fine-tune the semantic-similarity model on subject-specific corpora (for example, computer science or biology terminology) to further improve scoring accuracy for technical topics.
- **Mobile Application:** Package the core pipeline behind a lightweight API and build a dedicated mobile app for on-the-go practice.